

EECS 4314 E: Advanced Software Engineering  
**Project Proposal Document**  
**Group 4**

Github: <https://github.com/nurjahan-shiah/StudySync>

Nurjahan Ahmed Shiah  
Ali Shandhor  
Muhammad Fahad Sohail  
Saaram Ahmed Mustafa  
Uzma Alam  
Nusayba Hossain

Due: Friday, May 29, 2026

## Q1: Name of the product

---

# StudySync

## Q2: Brief description of the core concept of our application in terms of user functionality

---

StudySync is a dynamic, database-driven university collaboration platform designed to replace the fragmented tools students currently use, such as group chats, shared drives, and Discord servers, with a single, course-aware environment.

Our proposed platform supports three distinct user roles:

### **Students**

- Register, browse, and join study groups tied to enrolled courses.
- Attend or schedule study sessions with time and location details.
- Upload and access shared notes, PDFs, and other resources.
- Receive intelligent study group recommendations powered by a Machine Learning pipeline that analyses course enrollment and platform activity stored in the Data Lake.

### **Group Leaders**

- Create and manage study groups.
- Schedule sessions, post announcements, and assign tasks to members.
- Moderate shared resources within the group.

### **Admins**

- Oversee the entire platform, for example: manage users, courses, and groups.
- Handle moderation and maintain platform integrity.

### **Data Lake Integration: User-Facing Impact**

This layer:

- Ingests every user event, for example, file uploads, session joins, resource views, group interactions, into a centralized, schema-on-read store.

- Stores raw uploaded notes and PDFs alongside structured metadata, enabling future full-text search and content-based recommendations.
- Feeds a Python-based ML pipeline that surfaces personalized study group suggestions, reducing the friction of finding collaborators and increasing platform engagement.
- Supports ACID-compliant concurrent writes as students simultaneously upload resources and generate activity events.

### Q3: Group members - Group 4

---

1. Nurjahan Ahmed Shiah
2. Ali Shandhor
3. Muhammad Fahad Sohail
4. Saaram Ahmed Mustafa
5. Uzma Alam
6. Nusayba Hossain

### Q4. Programming languages and web development frameworks

---

#### Languages

- Python: backend API and business logic
- TypeScript: frontend, enforcing type safety across all React components
- SQL: database queries, schema design, and migrations

#### Frontend

- Next.js 16 (React): App Router, server-side rendering, protected routes
- Tailwind CSS: Utility-first styling
- TypeScript: Static typing throughout
- ESLint: Code quality enforcement

## Backend

- FastAPI: RESTful API, automatic Swagger docs, Pydantic validation
- SQLAlchemy: ORM for database models
- Alembic: schema migrations and version control
- Passlib + python-jose: password hashing and JWT authentication
- Uvicorn: ASGI server that runs the FastAPI application
- Python-dotenv: environment variable management

## Database

- PostgreSQL 16: Primary relational database  
*Transactional workloads (users, groups, sessions, resources)*

## Data Lake/Lakehouse Tech

- Delta Lake: Core Data Lakehouse storage layer such as: ACID transactions, schema enforcement and governance, decoupled storage and compute, diverse data type support
- Apache Parquet: Columnar file format underlying all Delta Lake tables.
- PyArrow: Arrow-based serialization layer required by Delta Lake for writing Parquet files.
- DuckDB: Lightweight in-process SQL engine for querying Delta Lake tables.
- pandas: Data manipulation in the ETL pipeline and ML feature engineering.
- scikit-learn: ML pipeline for course-overlap similarity and group recommendations.

## DevOps / Deployment

- Docker + Docker Compose: Multi-container orchestration, environment-agnostic builds
- Git + GitHub: Version control and team collaboration
- Supabase: Binary file storage for PDFs and images; PostgreSQL stores metadata links.

Q5.

- ❖ Rough diagram and description of our system architecture

- ❖ Explanation of which project requirements we strive to implement, and how this fulfils the learning goals for this course
- 

### **Architecture Pattern:** Service-Oriented Architecture (SOA) with a Dual-Tier Data Layer

StudySync follows a Service-Oriented Architecture (SOA) pattern. Rather than a single monolithic application or fully independent microservices, the system will be organized as a set of collaborating FastAPI services that share a common platform: a PostgreSQL operational database and a Delta Lake analytical layer.

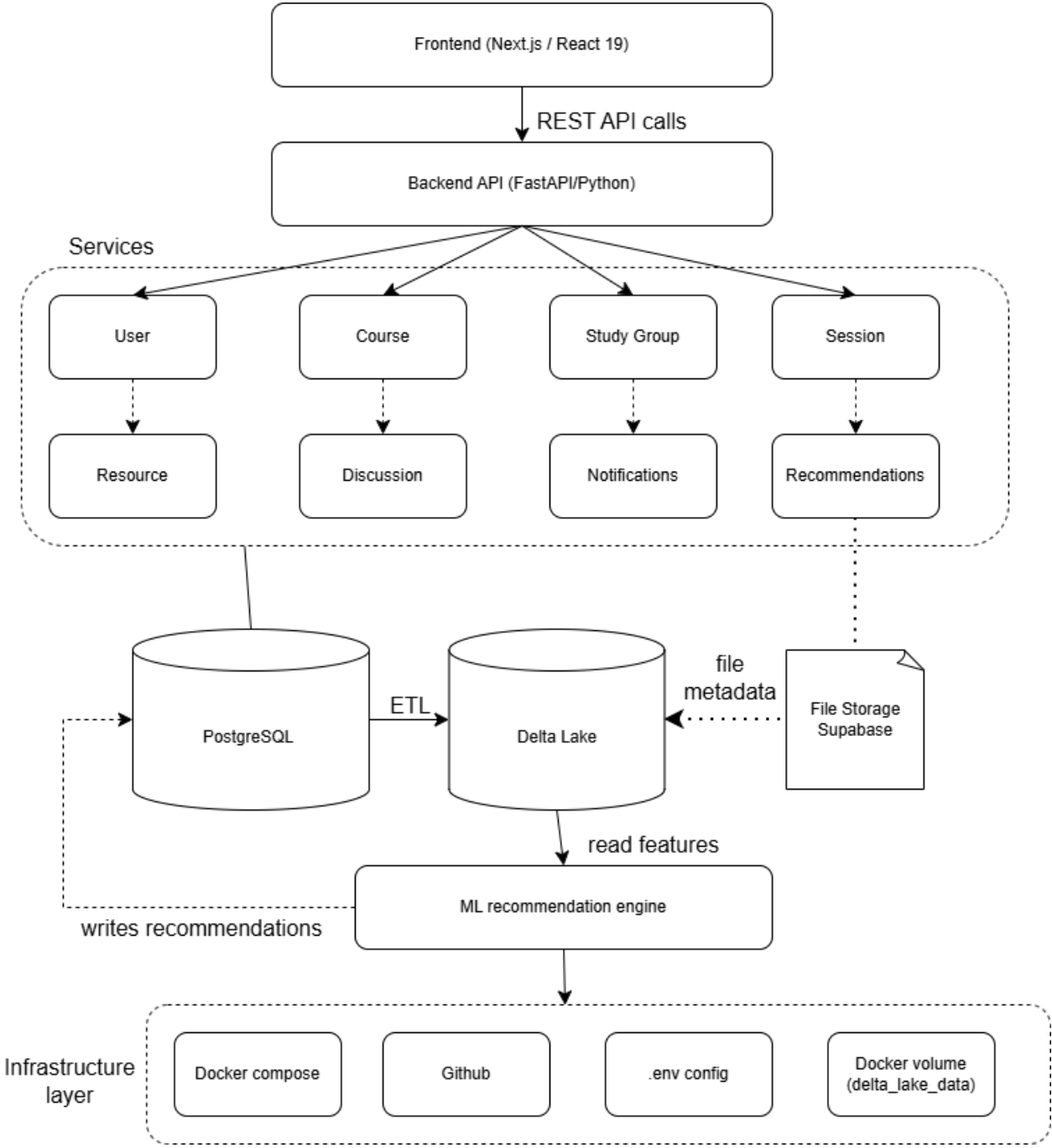
This pattern was chosen because:

- The 8 functional services (User, Course, Group, Session, Resource, Discussion, Notification, Recommendation) each own a distinct domain responsibility, which matches the SOA model of loosely coupled, reusable services.
- Unlike true microservices, the services share one PostgreSQL database and one FastAPI application.
- The addition of a Delta Lake analytical tier alongside PostgreSQL creates a dual-tier data layer: OLTP (operational) and analytical, the defining characteristic of a Data Lakehouse architecture.

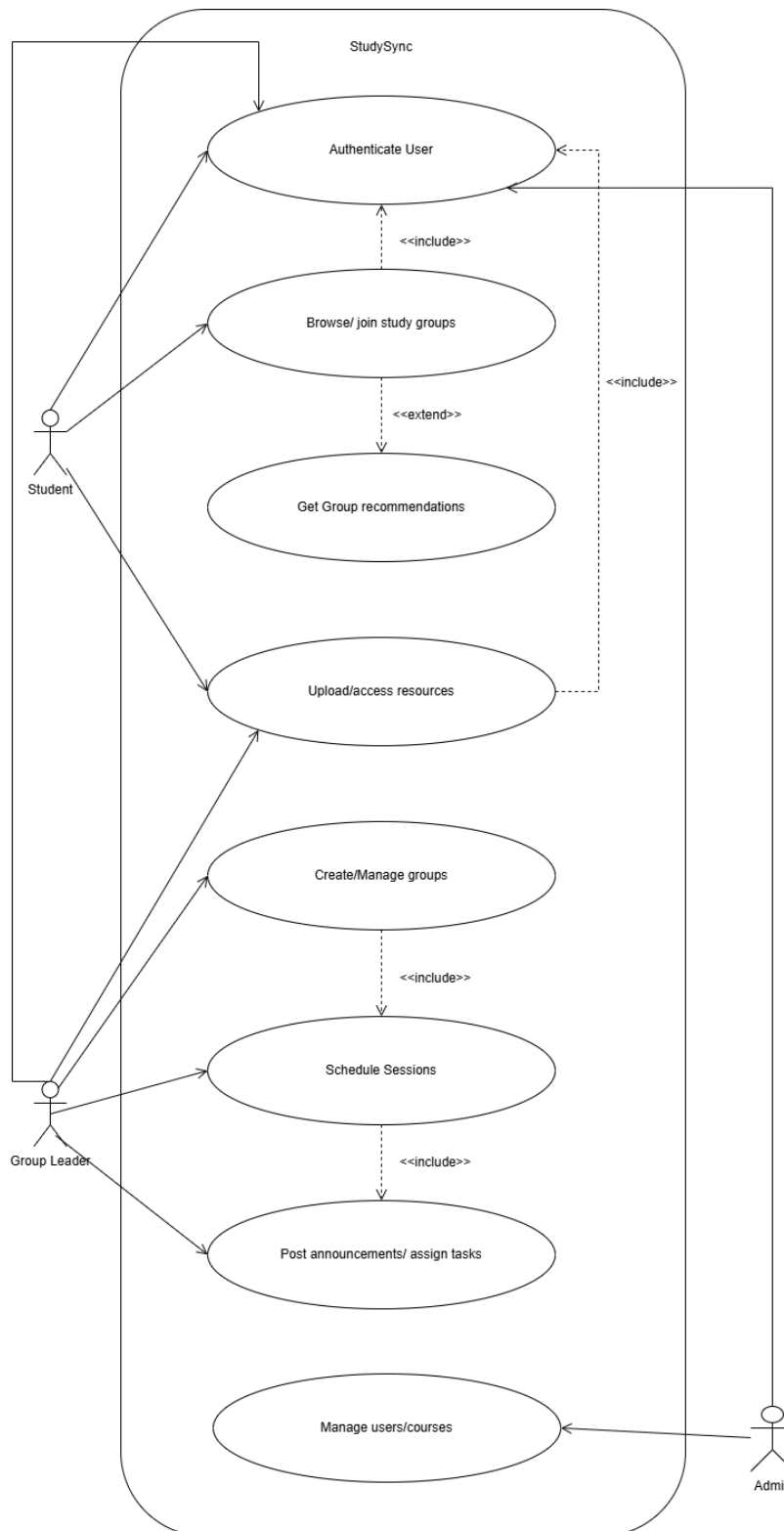
### **Four Architectural Layers:**

| Layer            | Components                                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Presentation     | Next.js 16 / React 19 / TypeScript / Tailwind CSS:<br>serves the UI, handles routing and protected pages                                     |
| Service          | FastAPI services: User/Profile; Course; Study Group; Session; Resource; Discussion; Notification; recommendation, each owns its domain logic |
| Data (Dual-Tier) | PostgreSQL 16 (OLTP: transactional writes) + Delta Lake (analytical: event logs, feature tables, ML pipeline)                                |
| Infrastructure   | Docker Compose; S3/Supabase file storage; GitHub version control                                                                             |

# System Architecture Diagram



## Use Case Diagram



Which project requirements do we strive to implement:

1. **User authentication and role-based access control:** JWT-based auth with three roles (Student, Group Leader, Admin), enforced in FastAPI middleware using python-jose and Passlib bcrypt.
2. **Relational database design:** Full ERD with foreign keys across all entities implemented via SQLAlchemy ORM, managed with Alembic migrations.
3. **CRUD operations:** Create, read, update, delete on all core entities (users, courses, groups, sessions, resources, tasks, announcements) via FastAPI endpoints with Pydantic validation.
4. **File upload and storage:** Both students and group leaders will be able to upload their files, such as PDF files, notes, pictures, etc., that will be stored in Supabase storage services, with the Database containing links to these files via the Resource Service.
5. **Notifications:** A notification service will be provided to notify users about new session schedules, assignments, and group announcements.
6. **Recommendation engine(Delta Lake):** ETL pipeline extracts activity events and file metadata into Delta Lake (Parquet + ACID). The ML pipeline will be used to read feature tables via DuckDB and scikit-learn to generate personalized study group recommendations.
7. **Deployment:** All five services (db, backend, frontend, etl, recommender) are orchestrated via Docker Compose for one-command deployment.

How does this fulfill the learning goals for EECS 4314:

1. **Software modelling:** StudySync requires us to create a system architecture diagram, a UML use case diagram, an ER diagram, and interaction diagrams such as a sequence diagram for the recommendation flow. This directly supports the course goal of deriving and expressing models of software systems using UML-style representations.

2. **Software architecture:** The project follows a Service-Oriented Architecture (SOA) with a dual-tier data layer. Rather than a monolith or microservices, eight FastAPI services each own a distinct domain responsibility and share a common platform. The proposal explicitly compares SOA against microservices and pure layered architecture and justifies the chosen pattern, addressing the course goal of understanding differences between types of software architecture.
3. **Data management:** StudySync implements a dual-tier data strategy: PostgreSQL 16 for normalized transactional data and Delta Lake for raw analytical data. This directly applies the Data Lake and Data Lakehouse concepts from the course.
4. **Security and risk management:** The system includes JWT authentication, role-based access control with three permission scopes (Student, Group Leader, Admin), protected Next.js routes, FastAPI middleware enforcement, ACID-compliant Delta Lake transactions to prevent data corruption, and admin moderation to manage risks such as unauthorized access, privacy issues, and inappropriate content sharing.
5. **Software development patterns:** The project applies modular SOA design by separating responsibilities into eight single-responsibility services: User/Profile, Course, Study Group, Session, Resource, Discussion, Notification, and Recommendation. The ETL pipeline and ML recommender are independently deployable Docker services, following the separation of concerns principle.
6. **Project management:** Our team will divide responsibilities across frontend, backend, database, and data lake components, manage source code through GitHub, use Docker Compose for reproducible, environment-agnostic deployment, integrate components across service boundaries, and deliver against the EECS 4314 project milestones.